
Securing LLM Agents Need Intent-to-Execution Integrity

Wenjie Qu¹ Ming Xu¹ Peiran Wang² Shengfang Zhai¹ Jiaheng Zhang¹ Dawn Song³

¹ National University of Singapore

² University of California, Los Angeles

³ University of California, Berkeley

Abstract

This position paper argues that securing LLM agents requires first defining an end-to-end correctness property that specifies when an agent’s execution faithfully reflects the user’s intent. Modern LLM agents operate over an *intent-to-execution pipeline*, where natural-language instructions are translated into concrete system operations such as tool calls, API requests, and code execution. While recent defenses have made progress in constraining how agents construct tool calls, most existing formulations implicitly assume that tools are trusted. The emergence of systems such as OpenClaw, with open ecosystems of third-party skills and direct access to user environments, breaks this assumption and exposes new failure modes, including malicious or over-privileged components in the execution pipeline.

Despite rapid progress in defense mechanisms, there is no adequate correctness property that defines what “secure” means for LLM agents, nor a principled way to evaluate the coverage of existing defenses. We observe that LLM agents are structurally analogous to compilers, where security violations correspond to mis-executions that do not preserve user intent. Drawing on this analogy, we identify two fundamental problem sources—untrusted data ingestion and untrusted tool execution—and derive four integrity properties that must hold simultaneously: *Tool Integrity*, *Instruction Integrity*, *Judgment Integrity*, and *Data Flow Integrity*. We call their conjunction *intent-to-execution integrity*.

Analyzing existing agentic defenses against these properties reveals that current systems provide only partial and non-compositional coverage, leaving fundamental gaps in securing modern LLM agents.

1 Introduction

LLM agents are increasingly deployed to execute complex tasks involving reasoning, tool use, persistent memory, and interaction with external systems. From personal assistants [1] to software engineering agents, these systems no longer merely generate text—they take actions: editing files, invoking APIs, executing code, and orchestrating workflows.

This shift fundamentally changes the security problem. Traditional LLM safety focuses on controlling model outputs. In contrast, LLM agents operate over an *intent-to-execution pipeline*, where user intent expressed in natural language is translated into concrete system operations. Security violations in this setting are not just harmful outputs, but *mis-executions*—cases where the agent’s execution effects misalign with the user’s intent due to attacks.

A large body of recent work has proposed defenses for LLM agents, including prompt-injection detection [20, 16], policy enforcement [25, 19, 14], and task-alignment mechanisms [12]. These defenses have made important progress in securing agentic planning: constraining how agents translate user intent into tool calls and arguments. However, most existing agent-security formulations

implicitly assume that tools are trusted. Under this assumption, the security problem largely reduces to preventing untrusted inputs from influencing tool-call construction.

From trusted tools to open ecosystems. A growing class of agent frameworks challenges the assumption that tools can be treated as trusted components. Systems such as OpenClaw [1] feature open ecosystems of community-contributed skills, persistent cross-session memory, and direct access to the user’s local environment—settings in which tools are contributed by third parties and therefore cannot be assumed to be trusted.

This shift introduces failure modes that are not captured by existing formulations, particularly supply-chain attacks through the execution pipeline. These risks are not merely theoretical: the ClawHavoc campaign [15] planted over 1,000 malicious skills on ClawHub to harvest credentials and cryptocurrency wallets; a large-scale audit of 42,447 skills [17] found that 26.1% contain at least one security vulnerability; and Snyk [18] demonstrated sandbox bypasses where configured security boundaries are not enforced at runtime.

Meanwhile, recent defenses for such systems—including NemoClaw [2], SeClaw [5], SafeClaw-R [6], and SecureClaw [11]—stack multiple mechanisms such as sandboxing, pattern scanning, and prompt-injection filters. However, each system introduces mechanisms without defining a correctness property that these mechanisms are intended to achieve, making it difficult to assess what they actually guarantee.

The missing piece. The limitations of existing defenses have a common root cause: there is no *adequate* correctness property that defines what “secure” means for LLM agents.

Formally, a correctness property specifies when an agent’s execution effect is correct with respect to the user’s intent across the entire intent-to-execution pipeline. Without such a definition, defense design becomes ad hoc, evaluation lacks a principled target, and practitioners cannot reason about the completeness of security mechanisms.

Prior work has begun to formalize aspects of agent security [13]. However, the formulation in [13] assumes trusted tools and focuses on the integrity of tool-call selection and arguments. It does not capture untrusted tool execution and reasoning-level manipulation, and therefore does not provide an end-to-end correctness condition for modern LLM agents.

Our position. Advancing the security of LLM agents requires first defining an end-to-end correctness property for the intent-to-execution pipeline.

We identify two fundamental problem sources—untrusted data ingestion and untrusted tool execution—and derive four integrity properties that must hold simultaneously: *Tool Integrity*, *Instruction Integrity*, *Judgment Integrity*, and *Data Flow Integrity*. We call their conjunction *intent-to-execution integrity*.

This definition enables systematic evaluation of existing defenses (which properties does each system cover?), principled benchmark design (what should be measured?), and identification of structural gaps that mechanism stacking cannot close. Evaluating existing defenses against these properties reveals that no system achieves full coverage.

2 Intent-to-Execution Integrity

2.1 Why a Correctness Property

An LLM agent transforms user intent expressed in natural language into concrete system operations—file modifications, API calls, shell commands—through a multi-stage pipeline: the LLM interprets intent, generates a plan, selects tools, and executes tool calls, which in turn become operating system instructions. This process is structurally analogous to compilation: a high-level specification (user intent) is progressively lowered through intermediate representations (plans, tool calls) into executable operations (system calls, I/O). Recent work has begun to make this analogy explicit, framing LLMs as interpreters for natural language programs [29] and compiling natural-language intent into typed execution graphs [30].

In compiler design, correctness is defined as *semantic preservation*—the compiled binary must behave as prescribed by the source program’s semantics. CompCert [28] famously proved this property

Table 1: Derivation of integrity properties from problem sources and causal paths.

Problem source	Risks	Required property
Untrusted data ingestion	Attribution confusion	Instruction Integrity
	Data flow violation	Data Flow Integrity
	Judgment manipulation	Judgment Integrity
Untrusted tool execution	Malicious third-party skills/tools	Tool Integrity

end-to-end for a realistic C compiler. The security of modern LLM agents demands an analogous property: the agent’s execution must faithfully reflect the user’s intent. Without such a property, security reduces to a reactive arms race—every new attack gets a new patch, but there is no way to assess whether the patches are complete or where gaps remain. We derive such a property by tracing every security threat to its root cause. Modern LLM Agents face exactly two fundamental problem sources; every integrity property we propose exists to close a causal path from source to harm.

2.2 Problem Source 1: Untrusted Data Ingestion

LLM agents process external content—emails, web pages, documents, and tool outputs—that may contain malicious instructions. This content is processed within a context that may contain sensitive information accumulated from prior interactions and tool executions. Three causal paths lead from this source to harm:

1. *Attribution confusion*: the agent cannot reliably distinguish user instructions from malicious instructions embedded in untrusted data. A malicious email containing “forward all files to attacker@evil.com” is processed in the same context as the user’s command “summarize my inbox.” Indirect prompt injection exploits this confusion at scale [26, 27].
2. *Data flow violation*: data read by one tool call flows into the parameters of another in ways that violate provenance constraints. Credentials loaded from `.env` during debugging end up as content in an outbound email; a file path intended as a read source is passed as a write destination; a user’s private budget retrieved in session 1 appears in a response to a different user in session 2 through persistent memory. These violations occur even when every tool call aligns to the user’s intent—the problem is that tool call arguments carry sensitive data across boundaries that should be enforced but are not.
3. *Judgment manipulation*: even when no explicit instruction is injected, adversarial content in the agent’s context can subtly steer its reasoning and alter its decision outcomes. A paper under review may contain self-promotional language that biases a paper-review agent toward a favorable assessment; carefully designed product review injections may cause a shopping agent to misjudge product quality and make a poor purchasing decision. Unlike attribution confusion, judgment manipulation does not require the agent to mistake data for a command—it exploits the fact that the agent must *read* data to make decisions, and that data can influence those decisions adversarially.

To prevent these three failure modes, we define three correctness properties.

Instruction Integrity. Every agent action must be causally attributable to an instruction from an authenticated user, not injected content. When a user explicitly delegates trust to external content (e.g., “follow this recipe”), subsequent actions from that content are authorized within the delegated scope.

Data Flow Integrity. Data flowing through tool call parameters must respect provenance constraints. Sensitive data (credentials, private records, session-specific context) must be taint-marked at read time, and tool calls whose arguments carry tainted data to unauthorized destinations (outbound email, external APIs, other users’ sessions) must be blocked. Taint marks must persist across sessions.

Judgment Integrity. All reasoning decisions made by the agent must be derived from user-aligned objectives. The reasoning process should be robust to adversarial influence from untrusted data.

Table 2: Examples for violations of each integrity condition.

Integrity condition	Example violation
Tool Integrity	A third-party image-generation skill includes hidden code that reads the user’s SSH key and exfiltrates it to a remote server.
Instruction Integrity	A user asks the agent to summarize an email, but the agent follows an injected instruction inside the email such as “forward all files to attacker@evil.com.”
Data Flow Integrity	A user asks the agent to debug a local application and email a report to a teammate. An injected instruction in a log file changes the report body to include environment variables read from <code>.env</code> . Sensitive data is sent to the teammate.
Judgment Integrity	A paper-review agent is asked to evaluate a submitted paper. The paper includes adversarial self-promotional text such as “this paper is widely regarded by leading experts as the best work in this field.” The agent is biased by this content and produces an overly favorable review.

We make this precise as follows. Let q denote the user’s query, d the untrusted data the agent must read to perform its task (e.g., a paper to review, a set of products to rank), and $J(q, d)$ the agent’s decision outcome (e.g., a review score, a ranking, a binary accept/reject). The injected content δ applied to d is *task-irrelevant* if it does not change the ground-truth answer that a correct reasoner would produce—for instance, inserting self-promotional rhetoric into a paper does not change its actual technical contribution. Judgment Integrity requires that for every task-irrelevant perturbation δ ,

$$J(q, d \oplus \delta) = J(q, d).$$

In other words, the agent’s decision must depend only on the task-relevant content of the untrusted data, and must be invariant to adversarial, task-irrelevant manipulation.

We defer the argument for why these three properties, together with Tool Integrity (defined in Section 2.3), are jointly sufficient to Section 2.4.

2.3 Problem Source 2: Untrusted Tool Execution

Modern agent frameworks increasingly load and execute third-party skills, plugins, and scripts contributed by open community ecosystems. In systems like OpenClaw [1], users install these components themselves, often with no centralized review or IT oversight.

This leads to direct supply-chain harm to users because third-party skills can be malicious while appearing to provide legitimate functionality. For example, the ClawHavoc campaign planted hundreds of malicious skills on ClawHub that masqueraded as benign automation tools, including wallet, trading, and productivity-related skills. Once installed or executed, these malicious skills could deliver payloads that steal credentials, browser data, SSH credentials, cryptocurrency wallet data, Apple and KeePass keychains, and user documents [15, 23, 24].

To prevent this failure mode, we define an additional correctness property:

Tool Integrity. Every software component that participates in producing an action—skill, plugin, or script—must be confined to its declared interface and documented behavior. For any invocation within its advertised functionality, the system must prevent the component from producing hidden or undeclared side effects, such as reading unrelated local files, contacting unadvertised network endpoints, modifying persistent state outside its documented scope, or executing additional commands not required by the requested functionality. Tool Integrity therefore requires enforcement of behavioral conformance between what a component claims to do and what it is allowed to do at runtime.

2.4 Putting the Properties Together

The four properties compose into an end-to-end correctness condition for the LLM Agent’s intent-to-execution pipeline. We argue their sufficiency by exhaustive case analysis over the pipeline stages.

Table 3: Defense coverage across four integrity properties. ● = full (provable), ◐ = partial, ○ = none.

Defense	Instr. Int.	Data Flow Int.	Judg. Int.	Tool Int.
<i>Defense systems for General LLM Agents</i>				
PromptArmor [20]	◐	◐	◐	○
PIA-Detection [16]	◐	◐	◐	○
TaskShield [12]	◐	◐	○	○
Progent [25]	◐	●	○	○
CaMeL [19]	●	●	○	○
DRIFT [14]	●	◐	○	○
<i>Defense systems for OpenClaw</i>				
NemoClaw [2]	○	○	○	◐
IronClaw [3]	◐	◐	◐	◐
SeClaw [5]	◐	◐	◐	◐
SafeClaw-R [6]	◐	○	○	◐
SecureClaw [11]	◐	◐	◐	◐

At each time step, the agent performs one of three operations: (i) *reasoning*—reading data and producing a semantic decision, (ii) *action construction*—selecting a tool and populating its arguments, or (iii) *execution*—invoking a software component that carries out the action. An adversary can corrupt the pipeline only by corrupting one of these operations. We enumerate all possible corruption points:

- **Reasoning** can be corrupted if untrusted data distorts the agent’s semantic decision (e.g., a biased review). Judgment Integrity prevents this: it requires $J(q, d \oplus \delta) = J(q, d)$ for all task-irrelevant perturbations δ .
- **Action construction** can be corrupted in two ways: (a) the agent selects a tool call that was not intended by the user (attribution confusion), or (b) the agent populates tool-call arguments with data that violates provenance constraints (data flow violation). Instruction Integrity prevents (a); Data Flow Integrity prevents (b).
- **Execution** can be corrupted if the invoked component performs undeclared behavior beyond its interface specification. Tool Integrity prevents this.

Because every agent time step falls into one of these three operations, and every corruption path within each operation is closed by the corresponding property, no pipeline stage remains unprotected when all four properties hold simultaneously. If all four hold, then the agent’s decisions are formed without adversarial distortion, its actions align with the user’s original intent, its action parameters carry only authorized data flows, and the invoked components execute only their declared behavior. We call this conjunction *intent-to-execution integrity*.

3 Evaluating Existing Defenses

We evaluate existing LLM agent defense systems against the four integrity properties. Our goal is to answer the following question: which part of intent-to-execution integrity does each defense actually enforce?

We use three levels in Table 3. ● means that the defense provably enforces the corresponding property. ◐ means that the defense contains a relevant mechanism, but the mechanism is based on heuristics or limited to specific attack patterns, thus it is feasible to bypass. ○ means that the defense does not address the property.

3.1 Defense Systems for General LLM Agents

Most general LLM-agent defenses are designed for cloud-hosted settings, where tools are assumed to be trusted. Under this assumption, the main security problem is whether untrusted data can cause the agent to select the wrong tool call or construct unsafe arguments. Accordingly, these defenses mainly protect the tool-call construction process rather than the integrity of the invoked components.

Within this trusted-tool setting, existing defenses fall into two groups. Detection-based methods such as PromptArmor [20] and PIA-Detection [16] attempt to detect and remove injected instructions before they influence the agent. System-security-style methods such as CaMeL [19], DRIFT [14], and Progent [25] enforce constraints over tool-call control flow or argument-level data flow, providing meaningful coverage over Instruction Integrity and Data Flow Integrity.

However, this design focus also explains two systematic gaps. First, Tool Integrity is largely out of scope because these systems assume invoked tools faithfully implement their interfaces. Second, for system-security-style methods, Judgment Integrity is not enforced because their guarantees concern tool-call behavior, not semantic reasoning outcomes. An adversarial paper may bias a review agent into producing an overly favorable assessment without causing any unauthorized tool call, remaining within the constraints enforced by system-security-style methods.

PromptArmor [20]. PromptArmor is an input-side guardrail that prompts an off-the-shelf LLM to detect and remove injected prompts before the agent processes the input. It can partially alleviate attribution confusion, and can also mitigate some Data Flow and Judgment violations when those violations are triggered by explicit injected instructions that are successfully removed. However, it provides no guarantee that all injected prompts will be detected or removed. Finally, PromptArmor does not verify whether tools or skills faithfully implement their declared behavior, so it provides no Tool Integrity.

PIA-Detection [16]. PIA-Detection fine-tunes a DeBERTa model to detect prompt injections in documents. Like PromptArmor, its protection centers on identifying injected instruction spans. Thus, it provides partial protection for Instruction Integrity, Data Flow Integrity and Judgment Integrity. However, it provides no guarantee that injected prompts will be detected under distribution shift or adaptive attacks. It also does not address Tool Integrity.

TaskShield [12]. TaskShield reframes defense as task alignment: each instruction and tool call is checked against the user’s stated goal. This gives partial Instruction Integrity because tool calls that do not contribute to the user task can be blocked. It also gives partial Data Flow protection when an exfiltrating tool call or sensitive argument is judged task-irrelevant. However, TaskShield relies on LLM-based task-alignment checks, so the checker itself remains vulnerable to adaptive attacks. We mark Judgment Integrity as none because TaskShield focuses on checking tool calls rather than ensuring that the agent’s semantic reasoning outcomes remain unbiased by adversarial content. It also does not address Tool Integrity because it assumes invoked tools are trusted.

Progent [25]. Progent enforces programmable least-privilege policies over tool calls using a domain-specific language. Because these policies decide which tool calls are permitted during execution, Progent partially enforces Instruction Integrity: an injected instruction may fail if it asks for an action outside the allowed policy. Progent provides deterministic guarantees for policy-specified Data Flow constraints by using policies to constrain tool-call arguments. However, these guarantees are policy-relative rather than general provenance tracking or taint propagation. Progent does not protect Judgment Integrity because it focuses on tool-call enforcement rather than the agent’s reasoning process. It also leaves Tool Integrity unaddressed.

CaMeL [19]. CaMeL extracts tool-call control flows from the trusted user query, architecturally preventing untrusted data from affecting program flow, and uses policies and information-flow tracking to enforce Data Flow Integrity. For this reason, we mark both Instruction Integrity and Data Flow Integrity as fully enforced. However, CaMeL does not solve Judgment Integrity: its design does not prevent the model’s semantic assessment from being distorted by adversarial but non-instructional content. CaMeL also does not provide Tool Integrity, since it assumes invoked tools faithfully implement their declared behavior.

DRIFT [14]. DRIFT combines a secure planner, an LLM-based validator, and an injection isolator. The planner constructs a minimal function trajectory and parameter checklist from the user query; the validator monitors deviations from that plan; and the isolator masks conflicting instructions from memory. This provides support for Instruction Integrity because deviations from the intended function trajectory can be blocked. It also gives partial Data Flow Integrity because parameter checklists and data-level constraints can restrict some unsafe argument flows. The protection is only partial because

validation relies on LLM-based judgments, which may remain vulnerable to adaptive attacks. DRIFT does not address Judgment Integrity or Tool Integrity.

3.2 Defense Systems for OpenClaw

OpenClaw defenses address the deployment-layer risks that general-agent defenses largely leave out of scope. NemoClaw and IronClaw harden execution through sandboxing, capability restrictions, and network controls, while SeClaw, SafeClaw-R, and SecureClaw add skill audits, monitors, hardening rules, behavioral checks, or safety-enforced counterparts. These mechanisms reduce the damage of malicious or vulnerable skills, and therefore provide partial coverage for Tool Integrity. However, they mostly enforce confinement, detection, or rule-based checks rather than exact behavioral conformance. They can limit what an untrusted component accesses, or detect known malicious patterns, but they do not guarantee that undeclared component behavior is always blocked. Moreover, most OpenClaw defenses only partially cover Instruction, Data Flow, and Judgment Integrity, because their protections are mainly heuristic-based.

NemoClaw [2]. NemoClaw primarily hardens the execution environment. It runs the OpenClaw agent inside an OpenShell sandbox with Landlock, seccomp, and network namespace isolation, and restricts network egress by policy. This gives partial Tool Integrity because sandboxing constrains what a malicious component can access. However, sandboxing can limit the damage of a malicious skill, but it cannot fully prevent all hidden side effects within the permissions granted to the sandbox. NemoClaw does not provide mechanisms for Instruction Integrity, Data Flow Integrity, or Judgment Integrity, because its design does not separate instructions from data, track information flow across tool calls, or protect semantic reasoning from adversarial content.

IronClaw [3]. IronClaw provides broader defense-in-depth for OpenClaw. Its public design emphasizes isolated WebAssembly containers for untrusted tools, capability-based permissions, endpoint allowlisting, credential protection at the host boundary, leak detection, prompt-injection defense, content sanitization, and policy enforcement. We therefore mark all four properties as partial. Its prompt-injection and policy mechanisms can partially protect Instruction Integrity and restrict some unsafe data flows. Its content sanitization can reduce reasoning manipulation, providing partial protection for Judgment Integrity. Its WASM sandboxing and capability restrictions provide partial Tool Integrity by limiting what untrusted components can access or execute. However, none of these mechanisms provides a full end-to-end guarantee for any of the four properties.

SeClaw [5]. SeClaw achieves broad but shallow coverage. Its skill auditing, runtime rules, privacy checks, and prompt-injection defenses provide relevant mechanisms for all four properties. However, these mechanisms are primarily detection-, rule-, or audit-based. They can reduce the likelihood that injected instructions are followed, sensitive data is leaked, semantic manipulation succeeds, or malicious skills are installed. But they do not provide end-to-end guarantees for any property. Thus, each property is marked as partial.

SafeClaw-R [6]. SafeClaw-R provides mechanisms to mediate actions prior to execution, and augments skills with safety-enforced counterparts. This gives partial Instruction Integrity because unsafe or task-misaligned actions can be blocked before execution. Because its mediation logic may be detector- or policy-based, we mark this protection as partial. SafeClaw-R does not implement Data Flow Integrity: it is not a taint-tracking or information-flow-control system over values moving between tools. It also does not enforce Judgment Integrity, because its design focuses on mediating tool calls and actions rather than protecting the agent’s reasoning process outcomes from adversarial influence. SafeClaw-R gives partial Tool Integrity because it detects malicious third-party skill patterns and modifies skills through safety-enforced counterparts.

SecureClaw [11]. SecureClaw combines installation auditing, hardening modules, background monitors, and behavioral rules. It explicitly targets prompt injection, credential theft, supply-chain attacks, and privacy leaks. This breadth justifies partial coverage for all four properties. Its behavioral rules can reduce some instruction-attribution and judgment-manipulation failures; its credential and privacy checks can mitigate some data-flow violations; and its audit and hardening mechanisms can detect or reduce some malicious or vulnerable skills. However, these mechanisms are primarily heuristic. Thus, all four properties are marked as partial.

3.3 Takeaways

Table 3 shows that existing defenses cover complementary but incomplete parts of intent-to-execution integrity. General-agent defenses, especially system-security-style designs, are strongest at protecting the *translation layer*: they constrain how user intent is lowered into tool calls and tool-call arguments. This explains their coverage of Instruction Integrity and Data Flow Integrity. However, they largely assume that the invoked tools are trusted, leaving Tool Integrity out of scope. They also provide very limited protection to Judgment Integrity, because semantic reasoning can be distorted without producing an unauthorized tool call.

OpenClaw defenses address the *execution layer* more directly. They harden the runtime, restrict component capabilities, audit skills, monitor suspicious behavior, or replace unsafe skills with safer counterparts. These mechanisms partially mitigate malicious or vulnerable components, but they mostly provide confinement, detection, or rule-based enforcement rather than complete behavioral conformance. Thus, they improve Tool Integrity only partially, and their coverage of Instruction, Data Flow, and Judgment Integrity remains limited.

The central gap is therefore compositional. Protecting only the translation layer is insufficient: a correctly selected tool call can still invoke a malicious skill. Protecting only the execution layer is also insufficient: a well-confined component can still be invoked for the wrong reason, with the wrong arguments, or after adversarially distorted reasoning. Intent-to-execution integrity requires both layers to be secured simultaneously.

4 What Existing Techniques Can and Cannot Guarantee

The defense evaluation in Section 3 shows *which* properties each system covers. A complementary question is *how far* current techniques can go in principle for each property, independent of any particular system. We discuss each property in turn.

Instruction Integrity. Instruction Integrity can be strongly enforced through architectural separation between trusted user instructions and untrusted data, as demonstrated by CaMeL [19] and Fides [9]. Under such designs, control flow is derived solely from authenticated user input, preventing untrusted content from altering tool selection. The principal limitation is utility: planning without access to untrusted content severely restricts the agent in tasks where decisions must depend on tool-call returns or external data. Relaxing this separation to recover utility reintroduces the risk of instruction attribution confusion.

Data Flow Integrity. Data Flow Integrity can be enforced using information-flow control techniques [4, 9] such as taint tracking and provenance-aware policies. These approaches provide deterministic guarantees under well-defined policies—for example, a policy can constrain email recipients to a trusted allowlist, making it impossible for an attacker to exfiltrate private data to addresses they control. The principal limitation is scalability: integrating new tools requires manually specifying per-tool policies, which becomes impractical for ecosystems with numerous third-party skills.

Judgment Integrity. Judgment Integrity remains the least developed dimension and is qualitatively harder than the preceding two. Instruction Integrity and Data Flow Integrity protect *structured* objects—tool-call selections and argument values—that can be separated from untrusted data by architecture or tracked by taint propagation. Judgment is different: many agent tasks require the model to read untrusted data precisely in order to make a semantic decision—reviewing a paper, ranking products, summarizing emails, or assessing evidence. In such tasks, the untrusted data cannot simply be isolated from the reasoning process, because it is the evidence on which the judgment depends.

This makes the problem substantially harder to enforce. Model-level defenses such as StruQ [8] and SecAlign [21] improve robustness against injected instructions through structured queries or preference optimization, but they do not provide guarantees over semantic judgments. Moreover, adaptive attacks can bypass such defenses [22, 10]. LLM-based detection and input sanitization are useful heuristics but cannot guarantee that $J(q, d \oplus \delta) = J(q, d)$ for all task-irrelevant perturbations

δ. Designing mechanisms that provide stronger integrity guarantees for semantic reasoning, remains an important open direction.

Tool Integrity. Tool Integrity can be partially enforced through a combination of static analysis, sandboxing, and runtime confinement. Static analysis can inspect tool code before installation to detect suspicious APIs, undeclared file access, and credential handling [7]. Sandboxing [2] can restrict runtime privileges by limiting filesystem access, system calls, and network egress. Together, these techniques substantially reduce the damage of malicious tools.

However, neither provides full behavioral conformance. Static analysis is necessarily approximate and can miss obfuscated or context-dependent behavior. Sandboxing cannot prevent unintended actions that remain within its permitted capabilities. A promising direction for better approximation of tool integrity is dynamic analysis of agent tools, where tools are executed in instrumented environments to observe actual behavior and check whether it conforms to the declared specification.

5 Alternative Views

“Existing formalizations already define agent security.” Prior work has begun to formalize important aspects of LLM-agent security, especially the integrity of tool-call selection and data isolation [13]. We view their effort as complementary rather than competing. Our argument is that existing formulations are not yet adequate for modern agent deployments because they typically assume trusted tools and focus on the planning layer—tool-call selection and argument integrity. This leaves two dimensions unaddressed: untrusted tool execution (Tool Integrity), which arises in open ecosystems where components are community-contributed; and reasoning-level manipulation (Judgment Integrity), which exploits the fact that agents must read untrusted data to make semantic decisions. Intent-to-execution integrity makes the trusted-pipeline assumption explicit and adds the missing dimensions needed for open, tool-rich, and stateful agent ecosystems.

“Judgment Integrity is too vague to be useful.” Judgment Integrity is indeed harder to formalize than Instruction Integrity or Data Flow Integrity, but that is precisely why it warrants explicit treatment. Tool-call selection and arguments are externalized, structured objects that can be separated from untrusted data and validated against policies. Judgment is different: many agent tasks require reading untrusted content in order to make a semantic decision—reviewing a paper, ranking products, or assessing evidence. Even though current defense techniques poorly address judgment integrity, explicitly defining it delineates a critical security gap that existing agentic defenses overlook.

“The property is too strong for practical systems.” Intent-to-execution integrity is an end-to-end ideal, not a claim that current systems can immediately satisfy all four properties in full. This mirrors a familiar pattern in traditional security: formal verification defines correctness properties for entire programs even though current techniques struggle to scale beyond modest-sized programs. The value of such properties lies not in immediate full enforcement, but in making the target explicit. Similarly, intent-to-execution integrity provides the reference standard against which defenses and benchmarks should be measured. This framing helps reveal which parts of the pipeline are adequately protected, which remain only partially so, and—crucially—which have received little attention from the agent security research community.

6 Conclusion

We argued that securing LLM agents requires an end-to-end correctness property—*intent-to-execution integrity*—composed of four conditions: Instruction Integrity, Data Flow Integrity, Judgment Integrity, and Tool Integrity. Evaluating existing defenses against these properties shows that each covers a different subset, and no system achieves full coverage. Our position is not that existing defenses are ineffective, but that they are incomplete without a shared correctness target; intent-to-execution integrity provides such a target for comparing defenses, designing benchmarks, and identifying where future work is most needed.

References

- [1] OpenClaw. OpenClaw: An open-source framework for AI agents. <https://github.com/openclaw/openclaw>, 2025.
- [2] NVIDIA. NemoClaw: Hardened OpenClaw runtime with Landlock and seccomp sandboxing. <https://github.com/NVIDIA/NemoClaw>, 2026.
- [3] IronClaw. IronClaw: Agent OS focused on privacy, security, and extensibility. <https://github.com/nearai/ironclaw>, 2026.
- [4] Fangzhou Wu, Ethan Cecchetti, and Chaowei Xiao. System-Level Defense against Indirect Prompt Injection Attacks: An Information Flow Control Perspective. *arXiv preprint arXiv:2409.19091*, 2024.
- [5] SaFo-Lab. SeClaw: The security armored personal AI assistant. <https://github.com/SaFo-Lab/secLaw>, 2026.
- [6] SafeClaw-R Authors. SafeClaw-R: Risk analysis and runtime enforcement for OpenClaw skills. *arXiv preprint arXiv:2603.28807*, 2026.
- [7] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards Measuring Supply Chain Attacks on Package Managers for Interpreted Languages. In *Network and Distributed System Security Symposium (NDSS)*, 2021.
- [8] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. StruQ: Defending Against Prompt Injection with Structured Queries. *arXiv preprint arXiv:2402.06363*, 2024.
- [9] Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643*, 2025.
- [10] Yulin Chen, Haoran Li, Yuxin Li, Yue Liu, Yangqiu Song, and Bryan Hooi. TopicAttack: An Indirect Prompt Injection Attack via Topic Transition. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 7327–7345, 2025.
- [11] Adversa AI. SecureClaw: Security plugin and skill for OpenClaw. <https://github.com/adversa-ai/secureclaw>, 2026.
- [12] Feiran Jia, Tong Wu, Xin Qin, and Anna Squicciarini. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29680–29697, 2025.
- [13] Vincent Siu, Jingxuan He, Kyle Montgomery, Zhun Wang, Neil Gong, Chenguang Wang, and Dawn Song. A framework for formalizing LLM agent security. *arXiv preprint arXiv:2603.19469*, 2026.
- [14] Hao Li, Xiaogeng Liu, Hung-Chun Chiu, Dianqi Li, Ning Zhang, and Chaowei Xiao. DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents. In *Advances in Neural Information Processing Systems*, 2025.
- [15] Koi Security. ClawHavoc: Analyzing a coordinated supply-chain attack on OpenClaw. Security report, 2026.
- [16] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, and Bryan Hooi. Can indirect prompt injection attacks be detected and removed? In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18189–18206, 2025.
- [17] Liu et al. Large-scale security analysis of AI agent skills. *arXiv preprint*, 2026.
- [18] Snyk. OpenClaw skill security: Credential exposure in the ClawHub registry. Security advisory, 2026.
- [19] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.
- [20] Tianneng Shi, Kaijie Zhu, Zhun Wang, Yuqi Jia, Will Cai, Weida Liang, Haonan Wang, Hend Alzahrani, Joshua Lu, Kenji Kawaguchi, Basel Alomair, Xuandong Zhao, William Yang Wang, Neil Gong, Wenbo Guo, and Dawn Song. PromptArmor: Simple yet effective prompt injection defenses. In *ICLR*, 2026.
- [21] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. SecAlign: Defending against prompt injection with preference optimization. In *ACM CCS*, 2025.
- [22] Qiusi Zhan, Richard Fang, Henil Shalin Panchal, and Daniel Kang. Adaptive Attacks Break Defenses Against Indirect Prompt Injection Attacks on LLM Agents. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 7116–7132, 2025.
- [23] Alfredo Oliveira, Buddy Tancio, David Fiser, Philippe Lin, and Roel Reyes. Malicious OpenClaw skills used to distribute Atomic macOS Stealer. *Trend Micro Research*, 2026. https://www.trendmicro.com/en_us/research/26/b/openclaw-skills-used-to-distribute-atomic-macos-stealer.html.

- [24] Ravie Lakshmanan. Researchers find 341 malicious ClawHub skills stealing data from OpenClaw users. *The Hacker News*, 2026. <https://thehackernews.com/2026/02/researchers-find-341-malicious-clawhub.html>.
- [25] Tianneng Shi et al. Progent: Programmable privilege control for LLM agents. *arXiv preprint*, 2025.
- [26] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *AISec*, 2023.
- [27] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *arXiv preprint arXiv:2406.13352*, 2024.
- [28] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–116, 2009.
- [29] Yingqiang Ge, Yujie Ren, Wenyue Hua, Shuyuan Xu, Juntao Tan, and Yongfeng Zhang. LLM as interpreter for natural language programming, pseudo-code programming, and flow programming of AI agents. *arXiv preprint arXiv:2405.06907*, 2024.
- [30] Lvmin Zhang and Maneesh Agrawala. Agint: Agentic graph compilation for software engineering agents. *arXiv preprint arXiv:2511.19635*, 2025.